

CS543 · WEB SERVICES · ASSIGNMENT 4

Part D — Make the Orders Service Survive the Network

Selected CampusEats Service: Orders

Team ID: 17

Team Members: Anurag Tiwari (20251651028), Mohd Rashid (20251651057), Saurabh Singh (20251651089), Niraj Kumar Gupta (20251651066)

D1 · Call one other service

The Orders service will make one real HTTP call to the **Payments service** from Tutorial 4. This follows our Assignment 3 architecture, where payment processing is handled at the PayFast integration boundary. The Orders service will call the CampusEats Payments service rather than calling the external partner directly.

The Payments service address will be resolved from an environment variable named **PAYMENTS_SERVICE_URL**. No payment-service URL will be hard-coded in the Orders source code. For example, the runtime configuration may provide a value such as `http://payments:8080`.

Outbound request

Method	Endpoint	Purpose
POST	/payments	Request payment processing for an order.

D2 · Harden the outbound call

The Orders service will use a **5-second timeout** for the Payments HTTP call. A timeout prevents the request from waiting indefinitely when the dependency is slow or unreachable.

The call will use a limited retry policy with **exponential backoff and jitter**. A practical policy is at most three attempts, with delays based on $0.5 * 2^n$ seconds plus a small random jitter. This reduces the chance that many Orders requests retry at exactly the same time.

Only failures that are safe to retry will be retried. **4xx responses are never retried**, because they represent a client or domain error rather than a temporary network problem. If the payment operation is a create/charge operation, every retry carries the same **Idempotency-Key**, so the Payments service can recognize the repeated request and avoid charging twice.

Retry decision

Condition	Action
2xx success	Return the successful payment result to the Orders flow.
4xx client/domain error	Do not retry. Translate the error into the Orders service error contract.
Timeout / connection failure	Retry with exponential backoff + jitter, using the same Idempotency-Key.
5xx dependency failure	Retry only within the limited retry budget; then use the fallback.

D3 · Fallback when Payments is unreachable

If the Payments service is unreachable after the retry budget is exhausted, the Orders service will **fail the payment-dependent order operation** rather than create an order that appears successfully placed without confirmed payment. The service will return a consistent dependency-unavailable error to the client and the user can retry the order later.

Degrading by accepting the order as paid or pretending that payment succeeded would be unsafe because it could create an unpaid order or leave the system in an ambiguous payment state. Since Assignment 3 already treats payment faults such as **PAYMENT_GATEWAY_UNAVAILABLE** as an explicit domain error, failing closed keeps the Orders service consistent with the existing CampusEats error boundary.

Failure handling flow

1. Orders receives a request that requires payment processing.
2. Orders reads `PAYMENTS_SERVICE_URL` from the environment.
3. Orders sends the payment request with an Idempotency-Key and a 5-second timeout.
4. If a transient network/5xx failure occurs, Orders retries up to the configured limit using exponential backoff and jitter.
5. If Payments succeeds, Orders continues normally.
6. If Payments returns 4xx, Orders does not retry and returns the mapped domain error.
7. If Payments remains unreachable after retries, Orders returns `PAYMENT_GATEWAY_UNAVAILABLE` rather than degrading.

Connection with Assignment 3

Assignment 3 integrated the PayFast Payment Gateway using SOAP. The partner contract used a WSDL and exposed a *charge* operation, while CampusEats translated partner faults into its own domain errors. In Assignment 4, the Orders service keeps that boundary intact and communicates with the CampusEats Payments service over HTTP. The Orders service therefore does not need to know the PayFast SOAP fault vocabulary; it only handles the CampusEats payment result or the standardized dependency error.

Conclusion

The Orders service is hardened against common network failures by using an environment-configured dependency URL, a finite timeout, safe retries with exponential backoff and jitter, and an idempotency key for retried payment creation. When the dependency cannot be reached, the service fails safely instead of creating an order whose payment status is uncertain.